Guia de Estudo — Reunião com o Professor

Painel de Pessoas: TCC de People Analytics com IA

João Vitor

29 de setembro de 2026

Sumário

1. Como abrir a conversa (pitch de 30 segundos) 2

2. O núcleo da arquitetura: "A IA decide, o código sempre calcula" 3
2.1 O pipeline em 3 etapas . . . 3
2.2 Por que isso importa (fundamentação teórica) . . . 3

3. Justificativas das escolhas técnicas 4
3.1 Por que sem backend (tudo no navegador)? . . . 4
3.2 Por que usar uma IA existente (Gemini/Claude) em vez de treinar um modelo próprio? . . . 4
3.3 Por que React (e não só o Python/Streamlit)? . . . 4
3.4 Por que armazenamento local e não banco de dados na nuvem? . . . 4

4. "Qual a diferença disso pra uma IA genérica (ChatGPT etc.)?" 5
4.1 A resposta certa . . . 5
4.2 Pergunta específica: "Previsão e chat, uma IA genérica não faz isso também?" 5

5. Privacidade — as regras e por quê 7

6. A suíte de validação (eval) — os números que sustentam o trabalho 8

7. E se a base estiver incompleta? 9

8. Terminologia — atenção a este ponto 10

9. Login e múltiplos usuários — por que não tem (ainda) 11

10. Dicas de apresentação e o que NÃO prometer 12

11. Resumo de uma página (cola rápida antes de entrar) 13

1. Como abrir a conversa (pitch de 30 segundos)

Comece direto, sem enrolação. Algo assim:

> "O projeto é uma plataforma de People Analytics que recebe **qualquer** planilha de RH — não uma planilha com formato fixo — e usa um modelo de linguagem para entender a estrutura dela. Só que a IA **nunca calcula os números que aparecem no dashboard**. Ela só interpreta a planilha e propõe *o quê* calcular; quem calcula é sempre código determinístico. Isso resolve o principal problema de usar IA em dados sensíveis: alucinação numérica."

Se ele perguntar "por que isso é TCC e não só um projeto de programação?", a resposta é: o núcleo do trabalho é a *arquitetura* que separa interpretação (IA) de computação (código), e a validação empírica de que essa separação funciona (a suíte de avaliação, ver seção 6).

2. O núcleo da arquitetura: "A IA decide, o código sempre calcula"

Essa é a frase-chave. Repita ela sempre que puder — é o fio condutor de toda a defesa.

2.1 O pipeline em 3 etapas

1. **Interpretação (IA):** a IA recebe só os **cabeçalhos das colunas** e uma **amostra pequena de linhas** (nunca a base inteira) e devolve uma especificação declarativa em JSON: qual tabela é aquela (funcionários, desligamentos, folha, RS, entrevistas, movimentações), o que cada coluna significa, e sugestões de gráfico (fonte, filtro, agrupamento, métrica).
2. **Validação (código determinístico):** um validador em código confere se essa especificação faz sentido contra o schema real dos dados — se a coluna existe, se o tipo bate, se o agrupamento é permitido (por exemplo, nunca por nome ou CPF, ver seção 5).
3. **Cálculo (código determinístico):** só depois da validação, um motor de cálculo em código puro (sem IA nenhuma) processa os dados reais e produz os números exibidos.

Frase de defesa: "A IA nunca vê a base completa, e nunca faz conta. Ela só decide *o quê* mostrar; o *cálculo* de qualquer número é sempre feito por código auditável."

2.2 Por que isso importa (fundamentação teórica)

- **Alucinação de LLMs é um problema documentado** (Ji et al., 2023; Huang et al., 2025) — modelos de linguagem podem gerar números plausíveis, mas errados.
- A abordagem mais comum na literatura para mitigar isso é RAG (Retrieval-Augmented Generation) — dar à IA acesso a documentos/dados reais para fundamentar a resposta (Ayala & Béchard, 2024). É uma estratégia **preventiva**: tenta evitar que a alucinação aconteça.
- A estratégia deste projeto é diferente e **complementar**: em vez de confiar na IA para calcular corretamente mesmo com contexto, a arquitetura **impede estruturalmente** que a IA calcule qualquer coisa. É uma estratégia **corretiva/estrutural**, não preventiva — não depende da IA "se comportar bem", depende de um validador de código que é 100% determinístico e testável.

Se ele perguntar "por que não usar RAG então?": RAG ainda deixa a geração do número final na mão do modelo. Aqui, a geração do número final nunca passa pelo modelo — é uma garantia mais forte, ao custo de a IA ter menos flexibilidade (ela só escolhe entre operações pré-definidas do motor de cálculo).

3. Justificativas das escolhas técnicas

3.1 Por que sem backend (tudo no navegador)?

- O dado de RH é sensível. Sem servidor, o dado **nunca sai do computador do usuário** — nem para a nuvem da própria aplicação. Só os cabeçalhos e uma amostra pequena vão para o provedor de IA (Gemini/Claude), o resto do processamento é local.
- Simplifica implantação: é um site estático, não precisa de infraestrutura de servidor, banco de dados, autenticação de multi-usuário.
- Persistência local via IndexedDB — os projetos do usuário ficam salvos no navegador dele.

3.2 Por que usar uma IA existente (Gemini/Claude) em vez de treinar um modelo próprio?

- Treinar um modelo do zero para essa tarefa exigiria uma base de treinamento massiva e rotulada de planilhas de RH — inviável para um TCC e desproporcional ao problema.
- A tarefa da IA aqui é **bem delimitada**: interpretar estrutura de planilha e propor uma especificação declarativa. Modelos de linguagem generalistas já são muito bons nisso via *prompt engineering*, sem necessidade de fine-tuning.
- O ponto central do trabalho não é "treinar uma IA melhor", é a **arquitetura de validação** em torno de uma IA de propósito geral. Isso é o que garante confiabilidade, não o modelo em si.

3.3 Por que React (e não só o Python/Streamlit)?

- O React foi a escolha principal porque o produto final é pensado para **uso real por usuários de negócio**, no navegador, sem instalação — uma experiência de dashboard interativo fluida.
- O Streamlit foi construído como uma **segunda implementação, para comparação arquitetural** — não é o produto principal, é uma validação de que a arquitetura (IA decide / código calcula) funciona independente da stack, e serve de contraponto na seção de discussão do TCC (prototipagem rápida vs. experiência de produto).

3.4 Por que armazenamento local e não banco de dados na nuvem?

- Coerente com a decisão de não ter backend: os dados de RH ficam no computador de quem está analisando, nunca centralizados em um servidor de terceiros.
- Reduz superfície de risco de vazamento de dados sensíveis — não existe um banco central para ser invadido.

4. “Qual a diferença disso pra uma IA genérica (ChatGPT etc.)?”

Essa pergunta é praticamente garantida. A resposta **certa** (já ajustada depois de um retorno anterior do professor) evita duas armadilhas: (1) dizer que é “muito mais inteligente” (não é, usa o mesmo tipo de modelo por baixo) e (2) dizer que uma IA genérica “não consegue” fazer previsão ou chat (ela consegue, se você souber pedir).

4.1 A resposta certa

Uma IA genérica consegue fazer quase tudo isso, mas só se você pedir explicitamente, souber o que pedir, e montar o pipeline você mesmo a cada vez. O diferencial do projeto não é capacidade bruta da IA — é a **automação e a entrega pronta**, sem o usuário de negócio precisar saber o que perguntar ou como estruturar a análise.

Pontos concretos para usar na resposta:

- **Interpretação automática da planilha:** uma IA genérica não sabe, de cara, que a coluna “dt_admiss” é uma data de admissão e que aquela aba é a tabela de funcionários — alguém precisa dizer isso a ela, todas as vezes. Aqui isso é automático e reaproveitável.
- **Previsão/cenários:** um ChatGPT com Code Interpreter, se você pedir explicitamente “calcule a projeção de headcount pros próximos 6 meses com esses dados”, consegue gerar algo. Mas a pessoa de RH **precisa pensar em pedir isso**, saber que aquele tipo de análise existe e é possível. No painel, a aba de previsão aparece **automaticamente** quando há histórico suficiente — o usuário não precisa nem saber que “análise preditiva de headcount” é uma coisa que existe.
- **Chatbot:** da mesma forma, um chat genérico de IA existe separado dos seus dados — a pessoa teria que copiar/colar ou subir a planilha em cada conversa. Aqui o chat já **vem integrado dentro do dashboard**, com o contexto e as regras de privacidade (seção 5) aplicadas automaticamente em toda pergunta.
- **Confiabilidade numérica:** mesmo quando uma IA genérica calcula algo, ela calcula “por dentro” do próprio modelo — sem garantia estrutural de que o número está certo. Aqui, todo número exibido passa pelo motor de cálculo determinístico, nunca pela IA.

4.2 Pergunta específica: “Previsão e chat, uma IA genérica não faz isso também?”

Mesma lógica da seção 4.1: não negar que ela consegue — deslocar o argumento para automático vs. sob demanda.

Sobre previsão:

> “Uma IA genérica até consegue gerar uma previsão ou bater um papo com os dados, mas só se alguém souber pedir isso, na hora, prompt por prompt. A maioria das pessoas de RH nem vai pensar em pedir uma regressão de turnover — não é o tipo de coisa que passa pela cabeça de quem não é técnico. Na nossa plataforma, a página de Previsões já existe pronta, sem ninguém pedir nada —

é parte do produto entregue, não resposta a um prompt que alguém teve que formular certo."

Sobre o chat, especificamente:

> "O chat em si não é uma novidade — qualquer IA genérica já é um chatbot. A diferença é que o nosso já vem dentro do dashboard, com o contexto da base já carregado, sem a pessoa precisar copiar e colar dados numa conversa separada, sem precisar descrever a planilha de novo, e sempre calculando em cima dos dados reais (nunca 'estimando' uma resposta em texto, que é o risco de alucinação já explicado antes)."

Por que esse ponto é bom de levar: reforça que a barreira resolvida não é só "acesso a dados" — é também acesso à própria **ideia de que aquela análise é possível**. Uma pessoa de RH sem bagagem técnica não vai necessariamente saber que dá para pedir uma projeção estatística a um chat de IA, ou nem vai ocorrer a ela tentar. Ao entregar isso pronto, sem exigir que a pessoa saiba pedir, a plataforma reduz uma barreira que nem é técnica no sentido de "não sabe programar" — é uma barreira de **não saber o que perguntar**, o que está ainda mais alinhado com o problema central do trabalho.

Se ele perguntar: "então qualquer coisa que a plataforma faz, uma IA genérica também faria, só que manualmente?" — resposta honesta:

> "Na decisão, sim, parecido; na entrega, não — o ganho está em automatizar, integrar ao dashboard e garantir que o cálculo é sempre real, não em fazer algo tecnicamente impossível para uma IA genérica fazer isoladamente."

Frase-resumo pronta: "Não é que a IA genérica seja incapaz — ela é capaz, se alguém souber pedir e montar tudo manualmente a cada vez. O que este projeto faz é entregar isso de forma automática, integrada e com garantia estrutural de que os números estão certos, para uma pessoa de RH que não sabe (e não precisa saber) o que perguntar."

5. Privacidade — as regras e por quê

Provavelmente vai vir pergunta sobre como o sistema evita expor dado sensível de pessoas. As regras implementadas:

1. **Nunca revela o nome de um funcionário** — ativo ou desligado — mesmo se perguntado diretamente por nome ou matrícula.
2. **Nunca permite agrupar ou filtrar por nome, matrícula ou CPF** — isso é bloqueado no **código**, independente do que a IA sugerir. Ou seja, mesmo que a IA "erre" e proponha isso, o validador determinístico rejeita.
3. **Supressão de agregados sensíveis em grupos pequenos:** médias de salário/remuneração só são exibidas se o grupo (área, cargo etc.) tiver **pelo menos 5 pessoas** — abaixo disso, a informação individual ficaria implícita mesmo em uma média.
4. **Exceção deliberada:** o nome do **recrutador responsável** por uma vaga (dado de processo de recrutamento, não de uma pessoa/funcionário) pode ser informado livremente — não é dado pessoal restrito da mesma categoria.

Frase de defesa: "As regras de privacidade não dependem da IA se lembrar de aplicá-las — elas são reforçadas no código de validação, então mesmo que a IA erre, o sistema não deixa passar."

6. A suíte de validação (eval) — os números que sustentam o trabalho

Ele provavelmente vai perguntar "como você sabe que funciona?" — essa é a resposta com dados concretos.

- Foram criadas **7 bases sintéticas de RH**, com nomenclatura de colunas **deliberadamente diferente** das bases usadas durante o desenvolvimento (para não estar "viciado" nos mesmos nomes de coluna).
- Um gabarito manual (ground truth) foi definido para cada base.
- Um script automatizado (`run_evaluation.mjs`) faz chamadas reais à API da IA e mede 5 métricas:
  1. Acurácia em identificar se uma aba é dado de RH.
  2. Acurácia na classificação do papel da aba (funcionários, desligamentos, folha, RS, etc.).
  3. Recall do mapeamento de colunas.
  4. Precisão do mapeamento de colunas.
  5. Taxa de gráficos válidos gerados.
- **Resultado mais recente: 100% nas 5 métricas**, usando o modelo Claude Sonnet 5.

Frase de defesa: "Não é uma afirmação de que funciona — é uma medição, com bases que a IA nunca viu no desenvolvimento, e o resultado foi 100% nas cinco métricas avaliadas."

7. E se a base estiver incompleta?

Ponto forte a mencionar: o sistema foi adaptado para funcionar mesmo quando faltam tabelas do schema canônico (por exemplo, só recrutamento, ou só folha, ou recrutamento + desligamentos sem a tabela de funcionários). Nesses casos:

- As abas e gráficos que dependem de dados ausentes simplesmente **não aparecem** — não há erro, nem número inventado.
- A aba de "Previsões & Cenários" só aparece quando há **histórico de headcount suficiente** para gerar uma projeção com sentido.

Frase de defesa: "O sistema se adapta ao que existe na planilha. Se falta uma tabela, a análise que dependeria dela simplesmente não é oferecida — nunca é inventada."

8. Terminologia — atenção a este ponto

Correção já incorporada no trabalho, depois de um retorno anterior do professor: usar **“usuários de negócio”**, não **“usuários não técnicos”**. A distinção que sustenta essa escolha:

- “Não técnico” sugere ausência de competência técnica em geral — o que é impreciso e um pouco condescendente com quem tem profunda expertise de RH.
- “Usuário de negócio” comunica com precisão: a pessoa não tem formação em **programação ou modelagem de dados**, mas tem competência técnica plena na sua própria área (RH, gestão de pessoas).

Se ele perguntar sobre isso, é fácil: já foi corrigido no artigo e na apresentação, título e corpo do texto.

9. Login e múltiplos usuários — por que não tem (ainda)

Se perguntarem por que não existe autenticação/multi-usuário:

- O escopo do projeto, hoje, é **uma pessoa analisando os dados da própria organização, localmente** — não há necessidade de controle de acesso entre usuários porque não há usuários múltiplos compartilhando a mesma instância.
- Login faria sentido se o produto fosse **comercializado** como uma plataforma multi-tenant (várias empresas usando o mesmo sistema hospedado), quando aí sim entraria a necessidade real: controle de acesso e capacidade de **revogar** acesso de alguém, não só "conveniência" de logar.
- Isso pode ser mencionado como **trabalho futuro**, ligado a uma eventual versão comercial/hospedada.

10. Dicas de apresentação e o que NÃO prometer

- **Não superestime a IA.** Nunca diga que ela "calcula" ou "sabe" os números — ela **interpreta estrutura** e **propõe**; quem confirma e calcula é código.
- **Não diga que uma IA genérica "não consegue" fazer algo** que ela genuinamente consegue (ver seção 4). Foque em automação/entrega, não em capacidade.
- **Tenha a tela do projeto pronta para demonstrar** — subir uma planilha nova (idealmente uma que ele nunca viu) e mostrar o pipeline completo: upload → interpretação da IA → dashboard gerado → uma pergunta no chat.
- Se possível, demonstre com uma base **incompleta** (por exemplo, só recrutamento) para mostrar a adaptação — é um argumento forte e visual.
- Se ele perguntar algo que você não sabe, **não invente**. Diga que é um ponto que pode ser aprofundado em trabalhos futuros — é uma resposta legítima em banca.
- Fale devagar na explicação da arquitetura (seção 2) — é a parte mais importante e a que costuma gerar mais perguntas.

11. Resumo de uma página (cola rápida antes de entrar)

- **Frase-chave:** “A IA decide, o código sempre calcula.”
- **Pipeline:** IA interpreta (cabeçalho + amostra) → validador de código confere → motor de cálculo determinístico gera o número.
- **Por que não é só uma IA genérica:** automação + entrega integrada, não capacidade exclusiva. Previsão/chat: uma IA genérica até consegue, mas só se alguém souber pedir — aqui já vem pronto, sem o usuário precisar saber que aquela análise existe.
- **Privacidade:** nunca nome/salário individual; bloqueio de agrupamento por nome/CPF no código; supressão de agregados com menos de 5 pessoas; recrutador de vaga é exceção permitida.
- **Validação:** 7 bases sintéticas com colunas diferentes das de desenvolvimento, 100% nas 5 métricas.
- **Sem backend:** dado sensível não sai do navegador do usuário; só cabeçalho + amostra vai para a IA.
- **Bases incompletas:** o sistema se adapta, nunca inventa dado ausente.
- **Terminologia:** “usuários de negócio”, não “usuários não técnicos”.
- **Login:** fora do escopo atual; relevante só numa versão comercial multi-tenant, como trabalho futuro.